파싱 모듈 상세 분석 문서

개요

Circom 컴파일러의 파싱 모듈은 `.circom` 소스 파일을 추상 구문 트리(AST)로 변환하는 핵심 구성 요소입니다. 이 문서는 LALRPOP 파서 생성기를 사용한 문법 정의, Include 파일 처리, 그리고 문법 설탕 제거 과정을 상세히 분석합니다.

2.1 LALRPOP 파서 구조 및 문법 정의 분석

LALRPOP 파서 개요

Circom은 LALRPOP(LR(1) Automatic Left-to-right Rightmost derivation Parser generator)를 사용하여 파서를 생성합니다. `parser/src/lang.lalrpop` 파일에 정의된 문법 규칙이 컴파일 시점에 Rust 코드로 변환됩니다.

문법 파일 구조 (`lang.lalrpop`)

1. 헤더 및 임포트 섹션

```
use num_bigint::BigInt;

use program_structure::statement_builders::*;

use program_structure::expression_builders::*;

use program_structure::ast::*;

use program_structure::ast_shortcuts::{self,Symbol,TupleInit};

use std::str::FromStr;

use lalrpop_util::ParseError;
```

```
use program_structure::ast::produce_report;

use program_structure::error_definition::Report;

use program_structure::error_code::ReportCode;



grammar<'err>(file_id: usize, errors:&'err mut Vec<Report>, field: &BigInt, flag
```

핵심 매개변수:

- `file_id` : 현재 파싱 중인 파일의 식별자
- `errors` : 파싱 중 발생한 에러를 수집하는 벡터
- `field` : 유한체 연산을 위한 소수 값
- `flag_no_init` : 초기화 없는 선언 허용 여부

2. 최상위 문법 규칙 (`ParseAst`)

```
pub ParseAst:AST = {

<s:@L> <pragmas:ParsePragma*> <includes:ParseInclude*> <definitions:ParseDefinit

=> { let (ast, mut ers) = AST::new(Meta::new(s,e), pragmas, includes, definition

errors.append(&mut ers);

ast

},

};
```

구조 분석:

- `@L` , `@R` : 위치 정보 캡처 (시작/끝 위치)

- `*` : 0개 이상 반복
- `?` : 선택적 요소
- 파싱 순서: Pragma → Include → Definition → MainComponent

3. Pragma 처리

```
ParsePragma:Pragma = {
// 버전 지정
<s:@L> "pragma" "circom" <v: Version> Semicolon <e:@R> => Pragma::Version(Meta::
// 커스텀 템플릿 허용
<s:@L> "pragma" "custom_templates" Semicolon <e:@R> => Pragma::CustomGates(Meta:
// 에러 처리
"pragma" <!> Semicolon => match <>.error {
ParseError::UnrecognizedToken { ref token, .. } => {
errors.push(produce_report(ReportCode::UnrecognizedPragma,token.0..token.2, file
Pragma::Unrecognized
}
_ => unreachable!(),
},
};
```

에러 복구 메커니즘:

- `<!>` : 에러 복구 규칙
- 인식되지 않은 토큰에 대해 적절한 에러 리포트 생성

- 파싱 중단 없이 계속 진행

4. Include 파일 처리

```
ParseInclude:String = {
"include" <STRING> Semicolon => <>,
"include" <!> Semicolon => {
match <>.error {
ParseError::UnrecognizedToken { ref token, .. } => {
errors.push(produce_report(ReportCode::UnrecognizedInclude, token.0..token.2, fi
}
_ => unreachable!(),
}
"".to_owned()
},
};
```

표현식 우선순위 계층

LALRPOP는 표현식의 우선순위를 계층적으로 정의합니다:

```
// 최고 우선순위: parallel 연산자
Expression14: Expression = {
<s:@L> "parallel" <expr: ParseExpression1> <e:@L>
=> build_parallel_op(Meta::new(s, e), expr),
```

```
}

// 삼항 연산자: ? :
Expression13 : Expression = {
<s:@L> <cond: Expression12> "?" <if_true: Expression12> ":" <if_false: Expressio
=> build_inline_switch_op(Meta::new(s,e),cond,if_true,if_false),
};

// 논리 OR: ||
Expression12 = InfixOpTier<ParseBoolOr,Expression11>;

// 논리 AND: &&
Expression11 = InfixOpTier<ParseBoolAnd,Expression10>;

// 비교 연산자: == != < > <= >=
Expression10 = InfixOpTier<ParseCmpOpCodes,Expression9>;

// 비트 OR: |
Expression9 = InfixOpTier<ParseBitOr,Expression8>;

// 비트 XOR: ^
Expression8 = InfixOpTier<ParseBitXOR,Expression7>;
```

```
// 비트 AND: &

Expression7 = InfixOpTier<ParseBitAnd,Expression6>;



// 시프트 연산자: << >>

Expression6 = InfixOpTier<ParseShift,Expression5>;



// 덧셈/뺄셈: + -

Expression5 = InfixOpTier<ParseAddAndSub,Expression4>;



// 곱셈/나눗셈: * / \ %

Expression4 = InfixOpTier<ParseMulDiv,Expression3>;



// 거듭제곱: **

Expression3 = InfixOpTier<ParseExp,Expression2>;



// 단항 연산자: - ! ~

Expression2 = PrefixOpTier<ParseExpressionPrefixOpcode,Expression1>;
```

신호 및 컴포넌트 선언 문법

신호 선언

```
SignalHeader : VariableType = {
```

```
"signal" <signal_type: ParseSignalType?> <tags_list: ParseTagsVector>

=> {

let signal_type = match signal_type {

None => SignalType::Intermediate,

Some(st) => st,

};

VariableType::Signal(signal_type, tags_list)

},

<signal_type: ParseSignalType> "signal" <tags_list: ParseTagsVector>

=> {

VariableType::Signal(signal_type, tags_list)

}

};



ParseSignalType: SignalType = {

"input" => SignalType::Input,

"output" => SignalType::Output

};
```

템플릿 정의

```
ParseDefinition : Definition = {

<s:@L> "template" <custom_gate: "custom"?> <extern_c: "extern_c"?> <parallel: "p

<name: IDENTIFIER> <args:@L> <arg_names: ParseParenthesisArguments?> <arge:@R>
```

```
<body: ParseBlock> <e:@R>

=> match arg_names {

None

=> build_template(Meta::new(s,e), name, Vec::new(), args..arge, body,

parallel.is_some(), custom_gate.is_some(), extern_c.is_some()),

Some(a)

=> build_template(Meta::new(s,e), name, a, args..arge, body,

parallel.is_some(), custom_gate.is_some(), extern_c.is_some()),

},

}
```

제약 조건 문법

Circom의 핵심인 제약 조건 문법:

```
ParseSubstitution : Statement = {

// 일반 할당: =

<s:@L> <variable: ParseExpression> <ops: ParseAssignOp> <rhe: ParseExpression> <

=> {if let Expression::Variable {meta, name, access} = variable {

build_substitution(Meta::new(s,e),name,access,ops,rhe)

} else{

build_mult_substitution(Meta::new(s,e),variable,ops,rhe)

}},

// 신호 할당: <--
```

```
<s:@L> <lhe: ParseExpression> "-->" <variable: ParseExpression> <e:@R>
=> {if let Expression::Variable {meta, name, access} = variable {
build_substitution(Meta::new(s,e),name,access,AssignOp::AssignSignal,lhe)
} else {
build_mult_substitution(Meta::new(s,e),variable,AssignOp::AssignSignal,lhe)
}},
// 제약 조건 할당: <==
<s:@L> <lhe: ParseExpression> "==>" <variable: ParseExpression> <e:@R>
=> {if let Expression::Variable {meta, name, access} = variable {
build_substitution(Meta::new(s,e),name,access,AssignOp::AssignConstraintSignal,l
} else{
build_mult_substitution(Meta::new(s,e),variable,AssignOp::AssignConstraintSignal
}},
// 제약 조건 등식: ===
<s:@L> <lhe: ParseExpression> "===" <rhe: ParseExpression> Semicolon <e:@R>
=> build_constraint_equality(Meta::new(s,e),lhe,rhe),
};
```

`run_parser` 함수 동작 과정

`parser/src/lib.rs` 의 `run_parser` 함수는 전체 파싱 프로세스를 조율합니다:

```
pub fn run_parser(
file: String,
```

```
version: &str,
link_libraries: Vec<PathBuf>,
field: &BigInt,
flag_no_init: bool
) -> Result<(ProgramArchive, ReportCollection), (FileLibrary, ReportCollection)>
```

단계별 처리 과정:

1. **초기화**

```
let mut file_library = FileLibrary::new();
let mut definitions = Vec::new();
let mut main_components = Vec::new();
let mut file_stack = FileStack::new(PathBuf::from(file));
let mut includes_graph = IncludesGraph::new();
```

2. **파일 스택 처리 루프**

```
while let Some(crr_file) = FileStack::take_next(&mut file_stack) {
// 파일 검색 및 로드
let (found, path, src, crr_str_file, reports) =
find_file(crr_file, ext_link_libraries.clone());
// 파일 라이브러리에 추가
let file_id = file_library.add_file(path.clone(), src.clone());
// 개별 파일 파싱
```

```
let program = parser_logic::parse_file(&src, file_id, field, flag_no_init)
.map_err(|e| (file_library.clone(), e))?;
}
```

3. **Include 처리**

```
for include in includes {
let path_include = FileStack::add_include(
&mut file_stack,
include.clone(),
&link_libraries.clone()
).map_err(|e| (file_library.clone(), vec![e]))?;
includes_graph.add_edge(path_include)
.map_err(|e| (file_library.clone(), vec![e]))?;
}
```

4. **버전 호환성 검사**

```
warnings.append(
&mut check_number_version(
path.clone(),
program.compiler_version,
parse_number_version(version),
).map_err(|e| (file_library.clone(), vec![e]))?
```

```
);
```

토큰화 과정 및 AST 생성 메커니즘

1. 전처리 단계 (preprocess 함수)

```
pub fn preprocess(expr: &str, file_id: FileID) -> Result<String, ReportCollect
let mut pp = String::new();
let mut state = 0; // 0: 일반, 1: 라인 주석, 2: 블록 주석
let mut loc = 0;
let mut block_start = 0;


let mut it = expr.chars();
while let Some(c0) = it.next() {
loc += 1;
match (state, c0) {
(0, '/') => {
// 주석 시작 감지
match it.next() {
Some('/') => {
state = 1; // 라인 주석
pp.push(' '); pp.push(' ');
}
Some('*') => {
```

```
block_start = loc;

state = 2; // 블록 주석

pp.push(' '); pp.push(' ');

}

Some(c1) => {

pp.push(c0); pp.push(c1);

}

None => {

pp.push(c0);

break;

}

}

}

(1, '\n') => {

pp.push(c0);

state = 0; // 라인 주석 종료

}

(2, '*') => {

// 블록 주석 종료 확인

if  let Some('/') = it.next() {

pp.push(' '); pp.push(' ');

state = 0;

}
```

```
}
_ => {
// 주석 내부는 공백으로 치환
for _i in 0..c.len_utf8() {
pp.push(' ');
}
}
}
}
}
```

2. LALRPOP 파서 호출

```
pub fn parse_file(src: &str, file_id: FileID, field: &BigInt, flag_no_init: bo
-> Result<AST, ReportCollection> {
let mut errors = Vec::new();
let preprocess = preprocess(src, file_id)?;

let ast = lang::ParseAstParser::new()
.parse(file_id, &mut errors, field, flag_no_init, &preprocess)
.map_err(|parse_error| match parse_error {
InvalidToken { location } =>
produce_generic_report(format!("{:?}", parse_error), location..location, file_id
UnrecognizedToken { ref token, .. } =>
```

```
produce_generic_report(format!("{:?}", parse_error), token.0..token.2, file_id),
ExtraToken { ref  token } =>
produce_generic_report(format!("{:?}", parse_error), token.0..token.2, file_id),
_ => produce_generic_report(format!("{:?}", parse_error), 0..0, file_id)
})?;


if !errors.is_empty() {
return Err(errors.into_iter().collect());
}


Ok(ast)
}
```

3. AST 노드 생성

LALRPOP 문법 규칙에서 사용되는 빌더 함수들:

- `build_template()` : 템플릿 정의 AST 노드 생성
- `build_function()` : 함수 정의 AST 노드 생성
- `build_substitution()` : 할당문 AST 노드 생성
- `build_constraint_equality()` : 제약 조건 등식 AST 노드 생성
- `build_call()` : 함수/템플릿 호출 AST 노드 생성
- `build_variable()` : 변수 참조 AST 노드 생성

에러 처리 및 복구

LALRPOP는 강력한 에러 복구 메커니즘을 제공합니다:

1. **에러 복구 규칙 (`<!>`)**

```
Semicolon:() = {
";" => (),
<s:@L> ! <e:@R> => errors.push(produce_report(ReportCode::MissingSemicolon, s..e
}
```

2. **부분 파싱 허용**

```
ParseStatement2 : Statement = {
// ... 정상 규칙들 ...
<s:@L> ! Semicolon <e:@R> => {
errors.push(produce_report(ReportCode::IllegalExpression, s..e, file_id));
build_block(Meta::new(s,e), Vec::new()) // 빈 블록으로 복구
}
};
```

3. **위치 정보 보존**

- `@L`, `@R` 매크로로 정확한 소스 위치 캡처
- `Meta::new(s,e)` 로 모든 AST 노드에 위치 정보 첨부
- 에러 리포팅 시 정확한 위치 표시 가능

이러한 파싱 구조를 통해 Circom은 복잡한 회로 정의 언어를 안정적으로 처리하고, 의미 있는 에러 메시지를 제공할 수 있습니다.

2.2 Include 파일 처리 및 의존성 관리 분석

Include 시스템 개요

Circom의 Include 시스템은 모듈화된 회로 개발을 지원하기 위해 설계되었습니다. `include_logic.rs` 모듈은 파일 스택 관리, 라이브러리 검색, 순환 의존성 탐지 등의 핵심 기능을 제공합니다.

FileStack 클래스 분석

구조 정의

```
pub struct FileStack {
current_location: PathBuf, // 현재 처리 중인 파일의 디렉토리
black_paths: HashSet<PathBuf>, // 이미 처리된 파일들 (중복 방지)
stack: Vec<PathBuf>, // 처리 대기 중인 파일들
}
```

핵심 메서드 분석

1. `new()` - 초기화

```
pub fn new(src: PathBuf) -> FileStack {
let mut location = src.clone();
location.pop(); // 파일명 제거하여 디렉토리 경로만 유지
FileStack {
current_location: location,
black_paths: HashSet::new(),
```

```
stack: vec![src]
}
}
```

동작 원리:

- 메인 파일을 스택에 추가
- 현재 위치를 메인 파일의 디렉토리로 설정
- 처리된 파일 목록 초기화

2. `add_include()` - Include 파일 추가

```
pub fn add_include(
f_stack: &mut FileStack,
name: String, // include할 파일명
libraries: &Vec<PathBuf>, // 라이브러리 검색 경로들
) -> Result<String, Report> {
let mut libraries2 = Vec::new();
libraries2.push(f_stack.current_location.clone()); // 현재 디렉토리 우선
libraries2.append(&mut libraries.clone()); // 추가 라이브러리 경로들
for lib in libraries2 {
let mut path = PathBuf::new();
path.push(lib);
path.push(name.clone());
let path = std::fs::canonicalize(path); // 절대 경로로 변환
match path {
```

```
Err(_) => {} // 파일이 없으면 다음 경로 시도

Ok(path) => {

if  path.is_file() {

if !f_stack.black_paths.contains(&path) {

f_stack.stack.push(path.clone()); // 스택에 추가

}

return Result::Ok(path.to_str().unwrap().to_string());

}

}

}

}

Result::Err(produce_report_with_message(ReportCode::IncludeNotFound, name))

}
```

검색 우선순위:

1. 현재 파일의 디렉토리 (`current_location`)
2. 명령행에서 지정된 라이브러리 경로들 (`-l` 옵션)
3. 표준 라이브러리 경로

중복 방지 메커니즘:

- `black_paths` 로 이미 처리된 파일 추적
- 동일한 파일의 중복 include 방지

3. `take_next()` - 다음 파일 가져오기

```rust
pub fn take_next(f_stack: &mut FileStack) -> Option<PathBuf> {
loop {
match f_stack.stack.pop() {
None => {
break None; // 더 이상 처리할 파일 없음
}
Some(file) if !f_stack.black_paths.contains(&file) => {
f_stack.current_location = file.clone();
f_stack.current_location.pop(); // 디렉토리만 유지
f_stack.black_paths.insert(file.clone()); // 처리됨으로 표시
break Some(file);
}
_ => {} // 이미 처리된 파일은 건너뛰기
}
}
}
```

처리 순서:

- 스택 기반 깊이 우선 탐색 (DFS)
- 마지막에 추가된 파일부터 처리
- 처리된 파일은 `black_paths` 에 추가하여 중복 방지

IncludesGraph 클래스 분석

구조 정의

```
pub struct IncludesNode {
pub path: PathBuf,
pub custom_gates_pragma: bool, // pragma custom_templates 사용 여부
}

#[derive(Default)]
pub struct IncludesGraph {
nodes: Vec<IncludesNode>, // 파일 노드들
adjacency: HashMap<PathBuf, Vec<usize>>, // 인접 리스트 (의존성 그래프)
custom_gates_nodes: Vec<usize>, // 커스텀 게이트 사용 노드들
}
```

핵심 메서드 분석

1. `add_node()` - 노드 추가

```
pub fn add_node(&mut self, path: PathBuf, custom_gates_pragma: bool, custom_g
self.nodes.push(IncludesNode { path, custom_gates_pragma });
if custom_gates_usage {
self.custom_gates_nodes.push(self.nodes.len() - 1);
}
}
```

매개변수 의미:

- `custom_gates_pragma` : 파일에 `pragma custom_templates` 선언 여부
- `custom_gates_usage` : 파일에서 실제로 커스텀 게이트 사용 여부

2. `add_edge()` - 의존성 간선 추가

```
pub fn add_edge(&mut self, old_path: String) -> Result<(), Report> {
let mut crr = PathBuf::new();
crr.push(old_path.clone());
let path = std::fs::canonicalize(crr)
.map_err(|_e| produce_report_with_message(ReportCode::FileOs, old_path))?;
let edges = self.adjacency.entry(path).or_insert(vec![]);
edges.push(self.nodes.len() - 1); // 현재 노드를 타겟으로 하는 간선 추가
Ok(())
}
```

의존성 방향:

- `A include B` 인 경우, B → A 방향의 간선 생성
- 역방향 그래프로 구성하여 의존성 추적 용이

3. `get_problematic_paths()` - 문제 경로 탐지

```
pub fn get_problematic_paths(&self) -> Vec<Vec<PathBuf>> {
let mut problematic_paths = Vec::new();
for from in &self.custom_gates_nodes {
problematic_paths.append(&mut self.traverse(*from, Vec::new(), HashSet::new()))
```

```
}
problematic_paths
}
```

문제 상황:

- 커스텀 게이트를 사용하는 파일이 있음
- 하지만 해당 파일이나 그 의존성 체인에 `pragma custom_templates` 가 없음

4. `traverse()` - 의존성 체인 순회

```
fn traverse(
&self,
from: usize, // 시작 노드
path: Vec<PathBuf>, // 현재까지의 경로
traversed_edges: HashSet<(usize, usize)>, // 순환 방지용 간선 집합
) -> Vec<Vec<PathBuf>> {
let mut problematic_paths = Vec::new();
let (from_path, using_pragma) = {
let node = &self.nodes[from];
(&node.path, node.custom_gates_pragma)
};
let new_path = {
let mut new_path = path.clone();
new_path.push(from_path.clone());
new_path
```

```
};
if !using_pragma {
problematic_paths.push(new_path.clone()); // pragma 없는 경로 발견
}
if  let Some(edges) = self.adjacency.get(from_path) {
for  to  in  edges {
let  edge = (from, *to);
if !traversed_edges.contains(&edge) { // 순환 방지
let  new_traversed_edges = {
let  mut  new_traversed_edges = traversed_edges.clone();
new_traversed_edges.insert(edge);
new_traversed_edges
};
problematic_paths.append(&mut  self.traverse(
*to,
new_path.clone(),
new_traversed_edges,
));
}
}
}
problematic_paths
```

```
}
```

순환 의존성 탐지:

- `traversed_edges` 로 이미 방문한 간선 추적
- 동일한 간선을 다시 방문하면 순환으로 판단
- 무한 루프 방지

`find_file` 함수의 라이브러리 검색 로직

```
pub fn find_file(
crr_file: PathBuf, // 찾을 파일 경로
ext_link_libraries: Vec<PathBuf>, // 외부 라이브러리 경로들
) -> (bool, String, String, PathBuf, Vec<Report>) {
let mut found = false;
let mut path = "".to_string();
let mut src = "".to_string();
let mut crr_str_file = crr_file.clone();
let mut reports = Vec::new();
let mut i = 0;
while !found && i < ext_link_libraries.len() {
let mut p = PathBuf::new();
let aux = ext_link_libraries.get(i).unwrap();
p.push(aux);
p.push(crr_file.clone());
```

```
crr_str_file = p;

match  open_file(crr_str_file.clone()) {

Ok((new_path, new_src)) => {

path = new_path;

src = new_src;

found = true;

}

Err(e) => {

reports.push(e);

i = i + 1;

}

}

}

(found, path, src, crr_str_file, reports)

}
```

검색 전략:

1. 각 라이브러리 경로에서 순차적으로 파일 검색
2. 첫 번째로 발견된 파일 사용
3. 모든 경로에서 실패하면 에러 리포트 수집하여 반환

순환 의존성 탐지 및 처리 방법

1. 탐지 메커니즘

```
// run_parser 함수 내에서
let mut errors: ReportCollection = includes_graph.get_problematic_paths().iter
Report::error(
format!(
"Missing custom templates pragma in file {} because of the following chain of in
path.last().unwrap().display(),
IncludesGraph::display_path(path)
),
ReportCode::CustomGatesPragmaError
)
).collect();
```

2. 에러 메시지 생성

```
pub fn display_path(path: &Vec<PathBuf>) -> String {
let mut res = String::new();
let mut sep = "";
for file in path.iter().map(|file| file.display().to_string()) {
res.push_str(sep);
let result_split = file.rsplit_once("/");
if result_split.is_some(){
res.push_str(result_split.unwrap().1); // 파일명만 표시
} else{
```

```
res.push_str(&file);
}
sep = " -> ";
}
res
}
```

출력 예시:

```
Missing custom templates pragma in file circuit.circom because of the following
```

3. 처리 방법

1. **컴파일 중단**: 순환 의존성이나 pragma 누락 시 컴파일 중단
2. **명확한 에러 메시지**: 의존성 체인을 명시하여 문제 위치 파악 용이
3. **부분 복구**: 가능한 한 많은 파일을 처리한 후 종합적인 에러 리포트 제공

Include 시스템의 특징

장점

1. **모듈화 지원**: 코드 재사용성 향상
2. **라이브러리 시스템**: 표준 라이브러리 및 외부 라이브러리 지원
3. **중복 방지**: 동일 파일의 중복 include 자동 방지
4. **명확한 에러 처리**: 의존성 문제를 명확히 진단

제한사항

1. **순환 의존성 금지**: 상호 참조하는 파일들 사용 불가

2. **pragma 일관성**: 커스텀 게이트 사용 시 모든 의존성에 pragma 필요

3. **파일 시스템 의존**: 컴파일 시점에 모든 파일이 접근 가능해야 함

이러한 Include 시스템을 통해 Circom은 대규모 회로 프로젝트의 모듈화된 개발을 지원하면서도, 의존성 관리의 복잡성을 효과적으로 처리합니다.## 2.

3 문법 설탕 제거 과정 분석

문법 설탕(Syntactic Sugar) 개요

Circom 언어는 개발자의 편의를 위해 여러 문법 설탕을 제공합니다. `syntax_sugar_remover.rs` 모듈은 이러한 편의 문법을 기본 문법으로 변환하여 후속 컴파일 단계에서 처리하기 쉽게 만듭니다.

`apply_syntactic_sugar` 함수 분석

함수 시그니처 및 전체 구조

```
pub fn apply_syntactic_sugar(program_archive: &mut ProgramArchive) -> Result<(

// 1. 메인 컴포넌트 검증

if program_archive.get_main_expression().is_anonymous_comp() {

return Result::Err(anonymous_general_error(

program_archive.get_main_expression().get_meta().clone(),

"The main component cannot contain an anonymous call".to_string()

));

}

let old_templates = program_archive.templates.clone();



// 2. 템플릿 처리
```

```
for (_name, t) in &mut  program_archive.templates {
let  old_body = t.get_body().clone();
check_anonymous_components_statement(&old_body)?;
let (new_body, component_decs, variable_decs, mut  substitutions) =
remove_anonymous_from_statement(&old_templates, &program_archive.file_library, o
if  let Statement::Block { meta, mut  stmts } = new_body {
let  mut  init_block = vec![
build_initialization_block(meta.clone(), VariableType::Var, variable_decs),
build_initialization_block(meta.clone(), VariableType::Component, component_decs
];
init_block.append(&mut  substitutions);
init_block.append(&mut  stmts);
let  new_body_with_inits = build_block(meta, init_block);
check_tuples_statement(&new_body_with_inits)?;
let  new_body = remove_tuples_from_statement(new_body_with_inits)?;
t.set_body(new_body);
}
}


// 3. 함수 처리
for (_, t) in &mut  program_archive.functions {
let  old_body = t.get_body().clone();
if  old_body.contains_anonymous_comp() {
```

```
return Result::Err(anonymous_general_error(
old_body.get_meta().clone(),
"Functions cannot contain calls to anonymous templates".to_string()
));
} else {
check_tuples_statement(&old_body)?;
let  new_body = remove_tuples_from_statement(old_body)?;
t.set_body(new_body);
}
}
Result::Ok(())
}
```

주요 변환 과정

1. 익명 컴포넌트 (Anonymous Components) 처리

익명 컴포넌트란?

Circom에서 익명 컴포넌트는 다음과 같은 편의 문법입니다:

```
// 편의 문법 (익명 컴포넌트)
output <== Multiplier()(a, b);


// 기본 문법으로 변환됨
```

```
component temp = Multiplier();
temp.a <== a;
temp.b <== b;
output <== temp.out;
```

변환 과정 (`remove_anonymous_from_expression`)

```
AnonymousComp { meta, id, params, signals, names, is_parallel } => {
let mut declarations = Vec::new();
let mut seq_substs = Vec::new();

// 템플릿 존재 확인
let template = templates.get(&id);
if template.is_none() {
return Result::Err(anonymous_general_error(
meta.clone(),
format!("The template {} does not exist", id)
));
}
// 고유한 컴포넌트 이름 생성
let id_anon_temp = id.to_string() + "_" +
&file_lib.get_line(meta.start, meta.get_file_id()).unwrap().to_string() +
"_" + &meta.start.to_string();
// 컴포넌트 선언 생성
```

```
if var_access.is_none() {
declarations.push(build_declaration(
meta.clone(),
VariableType::Component,
id_anon_temp.clone(),
Vec::new(),
true
));
} else {
// 루프 내부인 경우 배열 컴포넌트로 처리
declarations.push(build_declaration(
meta.clone(),
VariableType::AnonymousComponent,
id_anon_temp.clone(),
vec![var_access.as_ref().unwrap().clone()],
true
));
}

// 템플릿 인스턴스화
let call = build_call(meta.clone(), id.clone(), params.clone());
let exp_with_call = if is_parallel {
build_parallel_op(meta.clone(), call)
```

```
} else {
call
};
let access = if var_access.is_none() {
Vec::new()
} else {
vec![build_array_access(var_access.as_ref().unwrap().clone())]
};
let sub = build_substitution(
meta.clone(),
id_anon_temp.clone(),
access.clone(),
AssignOp::AssignVar,
exp_with_call
);
seq_substs.push(sub);


// 입력 신호 할당 처리
let mut inputs_to_assignments = BTreeMap::new();
if let Some(m) = names {
// 명명된 입력 처리
let inputs = template.unwrap().get_inputs();
for (operator, name) in m {
```

```
if  operator != AssignOp::AssignConstraintSignal {
return Result::Err(anonymous_general_error(
meta.clone(),
"Anonymous components only admit the use of the operator <==".to_string()
));
}
if  inputs.contains_key(&name) {
inputs_to_assignments.insert(name, (operator, signals[n_expr].clone()));
} else {
return Result::Err(anonymous_general_error(
meta.clone(),
format!("The template {} does not have an input signal named {}",
template.unwrap().get_name(), name)
));
}
}
} else {
// 순서대로 입력 처리
let  inputs = template.unwrap().get_declaration_inputs();
for (i, value) in  signals.iter().enumerate() {
inputs_to_assignments.insert(
inputs[i].0.clone(),
(AssignOp::AssignConstraintSignal, value.clone())
```

```
);
}
}
// 입력 신호 할당문 생성
for (name_signal, (operator, expr)) in  inputs_to_assignments {
let  mut  acc = if  var_access.is_none() {
Vec::new()
} else {
vec![build_array_access(var_access.as_ref().unwrap().clone())]
};
acc.push(Access::ComponentAccess(name_signal.clone()));
let (mut  declarations2, mut  stmts, new_exp) =
remove_anonymous_from_expression(templates, file_lib, expr, var_access)?;
seq_substs.append(&mut  stmts);
declarations.append(&mut  declarations2);
let  subs = Statement::Substitution {
meta: meta.clone(),
var: id_anon_temp.clone(),
access: acc,
op: operator,
rhe: new_exp
};
seq_substs.push(subs);
```

```
}
// 출력 표현식 생성
let outputs = template.unwrap().get_declaration_outputs();
if outputs.len() == 1 {
// 단일 출력
let output = outputs.get(0).unwrap().0.clone();
let mut acc = if var_access.is_none() {
Vec::new()
} else {
vec![build_array_access(var_access.as_ref().unwrap().clone())]
};
acc.push(Access::ComponentAccess(output.clone()));
let out_exp = Expression::Variable {
meta: meta.clone(),
name: id_anon_temp,
access: acc
};
Result::Ok((declarations, vec![Statement::Block { meta: meta.clone(), stmts: seq
} else {
// 다중 출력 (튜플)
let mut new_values = Vec::new();
for output in outputs {
let mut acc = if var_access.is_none() {
```

```
Vec::new()
} else {
vec![build_array_access(var_access.as_ref().unwrap().clone())]
};
acc.push(Access::ComponentAccess(output.0.clone()));
let  out_exp = Expression::Variable {
meta: meta.clone(),
name: id_anon_temp.clone(),
access: acc
};
new_values.push(out_exp);
}
let  out_exp = Tuple { meta: meta.clone(), values: new_values };
Result::Ok((declarations, vec![Statement::Block { meta: meta.clone(), stmts: seq
}
}
```

루프 내 익명 컴포넌트 처리

루프 내에서 익명 컴포넌트를 사용하는 경우 특별한 처리가 필요합니다:

```
// While 문 내에서의 처리
Statement::While { meta, cond, stmt } => {
let  id_var_while = "anon_var_".to_string() +
```

```
&file_lib.get_line(meta.start, meta.get_file_id()).unwrap().to_string() +
"_" + &meta.start.to_string();
let  var_access = Expression::Variable {
meta: meta.clone(),
name: id_var_while.clone(),
access: Vec::new()
};
// 루프 카운터 변수 생성
var_declarations.push(build_declaration(
meta.clone(),
VariableType::Var,
id_var_while.clone(),
Vec::new(),
false
));
// 카운터 초기화
subs.push(build_substitution(
meta.clone(),
id_var_while.clone(),
vec![],
AssignOp::AssignVar,
Expression::Number(meta.clone(), BigInt::from(0))
));
```

```
// 루프 본문에서 카운터 증가
let next_access = Expression::InfixOp {
meta: meta.clone(),
infix_op: ExpressionInfixOpcode::Add,
lhe: Box::new(var_access.clone()),
rhe: Box::new(Expression::Number(meta.clone(), BigInt::from(1))),
};
let subs_access = Statement::Substitution {
meta: meta.clone(),
var: id_var_while,
access: Vec::new(),
op: AssignOp::AssignVar,
rhe: next_access,
};
}
```

2. 튜플 (Tuple) 처리

튜플 문법

Circom은 다중 할당을 위한 튜플 문법을 지원합니다:

```
// 편의 문법 (튜플)
(a, b, c) <== SomeTemplate()(x, y);

// 기본 문법으로 변환됨
```

```
component temp = SomeTemplate();

temp.x <== x;

temp.y <== y;

a <== temp.out1;

b <== temp.out2;

c <== temp.out3;
```

튜플 검증 (check_tuples_statement)

```
fn check_tuples_statement(stm: &Statement) -> Result<(), Report> {

match stm {

Statement::IfThenElse { cond, meta, .. } => {

if cond.contains_tuple() {

Result::Err(tuple_general_error(

meta.clone(),

"A tuple cannot be used inside a condition".to_string()

))

} else {

// 재귀적으로 하위 문장 검사

check_tuples_statement(if_case)?;

if let Some(else_case) = else_case {

check_tuples_statement(else_case)?;

}
```

```
Result::Ok(())
}
}
Statement::While { meta, cond, .. } => {
if cond.contains_tuple() {
Result::Err(tuple_general_error(
meta.clone(),
"A tuple cannot be used inside a condition".to_string()
))
} else {
check_tuples_statement(stmt)
}
}
// 기타 문장 타입들에 대한 검증...
}
}
```

튜플 제거 (`remove_tuples_from_statement`)

```
fn remove_tuples_from_statement(stm: Statement) -> Result<Statement, Report> {
match stm {
Statement::MultSubstitution { meta, lhe, op, rhe } => {
let new_exp_lhe = remove_tuple_from_expression(lhe);
let new_exp_rhe = remove_tuple_from_expression(rhe);
```

```
match (new_exp_lhe, new_exp_rhe) {
(Expression::Tuple { values: mut values1, .. },
Expression::Tuple { values: mut values2, .. }) => {
if values1.len() == values2.len() {
let mut substs = Vec::new();
while values1.len() > 0 {
let lhe = values1.remove(0);
if let Expression::Variable { meta, name, access } = lhe {
let rhe = values2.remove(0);
if name != "_" {
substs.push(build_substitution(meta, name, access, op, rhe));
} else {
// 언더스코어 변수 처리 (값 무시)
substs.push(Statement::UnderscoreSubstitution {
meta: meta,
op,
rhe: rhe
});
}
} else {
return Result::Err(tuple_general_error(
meta.clone(),
"The elements of the receiving tuple must be signals or variables.".to_string()
```

```
));
}
}
return Result::Ok(build_block(meta.clone(), substs));
} else {
return Result::Err(tuple_general_error(
meta.clone(),
"The number of elements in both tuples does not coincide".to_string()
));
}
},
(lhe, rhe) => {
// 타입 불일치 에러 처리
if  lhe.is_tuple() || lhe.is_variable() {
return Result::Err(tuple_general_error(
rhe.get_meta().clone(),
"This expression must be a tuple or an anonymous component".to_string()
));
} else {
return Result::Err(tuple_general_error(
lhe.get_meta().clone(),
"This expression must be a tuple, a component, a signal or a variable".to_string
));
```

```
}
}
}
},
// 기타 문장 타입들에 대한 재귀적 처리...
}
}
```

중첩 튜플 평탄화 (`remove_tuple_from_expression`)

```
pub fn remove_tuple_from_expression(exp: Expression) -> Expression {
use Expression::*;
match exp {
Tuple { meta, values } => {
let mut unfolded_values = Vec::new();
for val in values {
let exp = remove_tuple_from_expression(val);
if let Tuple { values: mut values2, .. } = exp {
// 중첩된 튜플을 평탄화
unfolded_values.append(&mut values2);
} else {
unfolded_values.push(exp);
}
```

```
}
build_tuple(meta, unfolded_values)
},
_ => exp,
}
}
```

AST 변환 규칙 및 최적화 기법

1. 변환 규칙

익명 컴포넌트 → 명시적 컴포넌트

```
AnonymousComp(Template, params, signals)
→
{
component temp_N_L = Template(params);
temp_N_L.input1 <== signals[0];
temp_N_L.input2 <== signals[1];
// ...
return temp_N_L.output;
}
```

튜플 할당 → 개별 할당

```
(a, b, c) <== (x, y, z)
```

```
→
{
a <== x;
b <== y;
c <== z;
}
```

언더스코어 변수 처리

```
(a, _, c) <== SomeFunction()
→
{
a <== temp.out1;
_ <== temp.out2; // UnderscoreSubstitution
c <== temp.out3;
}
```

2. 최적화 기법

고유 이름 생성

```
let  id_anon_temp = id.to_string() + "_" +
&file_lib.get_line(meta.start, meta.get_file_id()).unwrap().to_string() +
"_" + &meta.start.to_string();
```

- 파일 라인 번호와 위치를 사용하여 충돌 없는 고유 이름 생성

초기화 블록 분리

```
let mut init_block = vec![
build_initialization_block(meta.clone(), VariableType::Var, variable_decs),
build_initialization_block(meta.clone(), VariableType::Component, component_decs
];
init_block.append(&mut substitutions);
init_block.append(&mut stmts);
```

- 변수/컴포넌트 선언을 블록 시작 부분으로 이동
- 할당문들을 적절한 순서로 배치

병렬 처리 지원

```
let exp_with_call = if is_parallel {
build_parallel_op(meta.clone(), call)
} else {
call
};
```

- `parallel` 키워드가 있는 경우 병렬 연산자로 래핑

에러 처리 및 검증

1. 제약 사항 검증

익명 컴포넌트 사용 제한

- 메인 컴포넌트에서 사용 금지

- 함수 내에서 사용 금지
- 할당문 좌변에서 사용 금지
- 조건문, 반복문 조건에서 사용 금지
- 배열 인덱스에서 사용 금지

튜플 사용 제한

- 조건문, 반복문 조건에서 사용 금지
- 함수 반환값으로 사용 금지
- 배열 인덱스에서 사용 금지
- 산술 연산에서 사용 금지

2. 에러 메시지 생성

```
pub fn anonymous_general_error(meta: Meta, msg: String) -> Report {
let mut report = Report::error(format!("{}", msg), ReportCode::IllegalExpressi
report.add_primary(meta.location.clone(), meta.get_file_id(), "here".to_string()
report
}

pub fn tuple_general_error(meta: Meta, msg: String) -> Report {
let mut report = Report::error(format!("{}", msg), ReportCode::IllegalExpressi
report.add_primary(meta.location.clone(), meta.get_file_id(), "here".to_string()
report
}
```

변환 결과

문법 설탕 제거 과정을 통해:

1. **코드 단순화**: 복잡한 편의 문법이 기본 문법으로 변환
2. **명시적 구조**: 모든 컴포넌트와 신호 연결이 명시적으로 표현
3. **최적화 준비**: 후속 컴파일 단계에서 처리하기 쉬운 형태로 변환
4. **에러 검출**: 잘못된 문법 사용을 조기에 발견하고 명확한 에러 메시지 제공

이러한 변환 과정을 통해 Circom은 개발자 친화적인 문법을 제공하면서도, 내부적으로는 일관되고 처리하기 쉬운 AST 구조를 유지할 수 있습니다.